iRAYPLE Acquisition Card

SDK Development Document (C Interface)



V1.3.0

Revision History

No.	Date	Version	Revision Content	Software Version
1	Oct. 2022	1.0.0	First Release	2.4.0
2	Oct. 2023	1.1.0	Added Routines and Interface Description of openInterfaceEx.	2.4.1
3	Apr. 2024	1.2.0	Added the Function of Save Image.	2.4.2
4	Nov. 2025	1.3.0	Added MVPImage Construction Method	2.6.1

Table of Contents

1 Overview **5**
1.1 Download 5
1.2 Compatibility Introduction 5
1.3 SDK Installation Directory 6
1.3.1 SDK for Windows 6
1.4 Related Resources of SDK (C interface) 7
1.5 SDK Engineering Configuration 8
1.5.1 VC6.0 8
1.5.1.1 Configure Header File 8
1.5.1.2 Configure lib 9
1.5.2 VS2005 ~ VS2013 10
1.5.2.1 Configure Header File 11
1.5.2.2 Configure lib 11
1.5.3 QT (Pro File) 13
2 Overall Calling Process of SDK **14**
3 Function Interface Calling of SDK **15**
3.1 System Operation Instruction 15
3.1.1 Obtain Version Information 15
3.1.2 Acquisition Card Enumeration (Find Device) 15
3.1.3 Camera Enumeration (Find Device) 16
3.2 Acquisition Card Operation 16
3.2.1 Open and Close Acquisition Card 16
3.3 Camera Operation Instruction 18
3.3.1 Device Open and Close 18
3.3.2 Registration Event Callback 19
3.4 Streaming Data Operation 20
3.4.1 Stream Data Configuration 20
3.4.2 Image Collection 21
3.4.3 Acquire Image Data 22
3.4.4 Operations on Streaming and Image Data 24
3.5 Attribute Operation Instruction 25
3.5.1 Configuration Download, Storage and Loading 25
3.5.2 Attribute Determination, Acquisition and Setting 26
3.5.2.1 Attribute Determination 26
3.5.2.2 Attribute Acquisition 27
3.5.2.3 Attribute Setting 31
3.5.3 Execution of Command Attribute 33
3.5.4 Attribute Search and Usage 33
3.5.4.1 Integer Attribute 33
3.5.4.2 Floating Point Attribute 36
3.5.4.3 Boolean Attribute 38
3.5.4.4 Enumeration Attribute 39

3.5.4.5 String Attribute....41
3.5.4.6 Command Attribute....42
3.6 Image Operation....42
3.6.1 Video Recording....42
3.6.2 Image Operation....43
4 Image Format of 3rd-party Platform Conversion....46
4.1 Halcon....46
4.1.1 Mono8 (Monochrome Image)....46
4.1.2 Non-Mono8 (Color Image)....46
4.2 OpenCV Object....47
4.2.1 Mono8 (Monochrome Image)....47
4.2.2 Non-Mono8 (Color Image)....47
4.3 QT Object....47
4.3.1 Mono8 (Monochrome Image)....47
4.3.2 Non-Mono8 (Color Image)....48
4.4 MvpImage Object....48
4.4.1 Mono8 (Monochrome Image)....48
4.4.2 Non-Mono8 (Color Image)....49
5 Matched Routine Description....1
6 FAQ (Frequently Asked Question)....2
6.1 Only 8 Images are Obtained by Using the C Interface....2

1 Overview

1.1 Download

You can download the SDK installation package on the official website:

http://download.huaraytech.com/pub/sdk/

Please download the appropriate version of SDK.

Figure 1-1 SDK installation packages



1.2 Compatibility Introduction

Table 1-1 Operating System Requirement

OS Type	Version	Note
Windows	Win7 SP1 32/64bit	Ensure that OS version includes the patch of SP1.
	Win8 32/64bit	
	Win10 32/64bit	

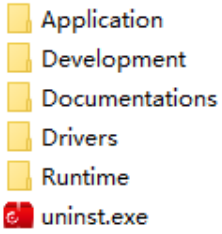
- Windows XP is no longer supported since SDK V2.2.5.
- The supported kernel version of Linux is 2.6.32 to 5.11.0 (Inclusive).
- Use the command of **uname –a** to check the kernel version number.

1.3 SDK Installation Directory

1.3.1 SDK for Windows

The installation directory structure of the SDK for Windows is as follows:

Figure 1-2 Installation directory structure



Since the V2.1.0 of SDK, user can select the installation path.

Table 1-2 Installation directory structure description

Application	● **MViewer** is compatible with 32-bit and 64-bit. ● **CamTools** is compatible with 32-bit and 64-bit.
Development	**3rdPartyPlatformAdapter** It includes two plug-in dynamic libraries of third-party algorithm platform (Halcon and Sherlock).
	DOTENT DLL It includes **.Net** library file which is imported when user create the project.
	Include It includes the header file of the SDK interface, which explains the basic properties of various interfaces. The specific usage of header files is in the **samples folder**. The other is the extension definition of the header file of the SDK, which defines the SDK error code, the user-defined parameters, the data restriction of some parameters, the enumeration of various behaviors, and the data structure of various information.
	Lib It includes the lib files.
	Samples

	It includes the various types of demo file, such as C#, Delphi, Lab View, MFC, Python, QT, VB and VC.
Documentations	● **.Net** interface document ● **ActiveX** interface document ● **C** interface document ● **SDK** development document
Drivers	● The installation and uninstallation software of **DShowFilter** high-performance driver. ● The installation and uninstallation software of **GigE** high-performance driver. ● The installation and uninstallation software of **PCIeCL** high-performance driver. ● The installation and uninstallation software of **USB** high-performance driver. ● The installation and uninstallation software of **CLVSP** high-performance driver.
Runtime	The compilation dynamic link library when running the SDK. It includes the third-party **GenICam** library which is separated into two folders as 32-bit and 64-bit.

1.4 Related Resources of SDK (C interface)

Figure 1-3 SDK source description

Category	Item	File Storage Folder
SDK Library	Header File	【SDK Installation Directory】 \Development\Include\IMVFG\IMVFGApi.h \Development\Include\IMVFG\IMVFGDefines.h
	Lib (32-bit)	【SDK Installation Directory】 \Development\Lib\win32\MVSDKmd.lib
	Lib (64-bit)	【SDK Installation Directory】 \Development\Lib\x64\MVSDKmd.lib
	Runtime Library (32-bit)	【SDK Installation Directory】\Runtime \Win32\MVSDKmd.dll
	Runtime Library (64-bit)	【SDK Installation Directory】\Runtime \x64\MVSDKmd.dll

C API Document	Chinese	【SDK Installation Directory】\Documentations\IMVFG_API_C_CHS.chm
	English	【SDK Installation Directory】\Documentations\IMVFG_API_C_ENG.chm
C Interface Routine	Path	【SDK Installation Directory】\Development\Samples\VC\IMVFG

1.5 SDK Engineering Configuration

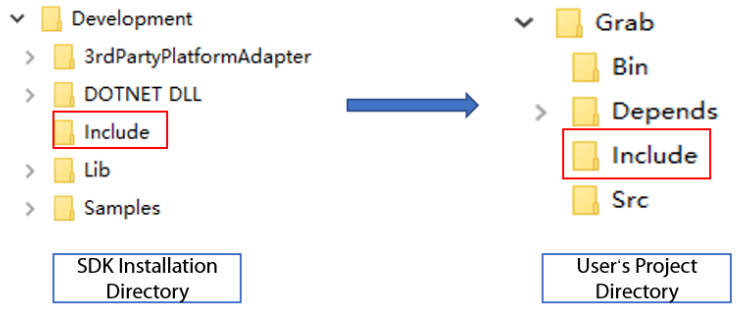
1.5.1 VC6.0

Take **Win32 | Debug** as an example to demonstrate the method of project configuration.

1.5.1.1 Configure Header File

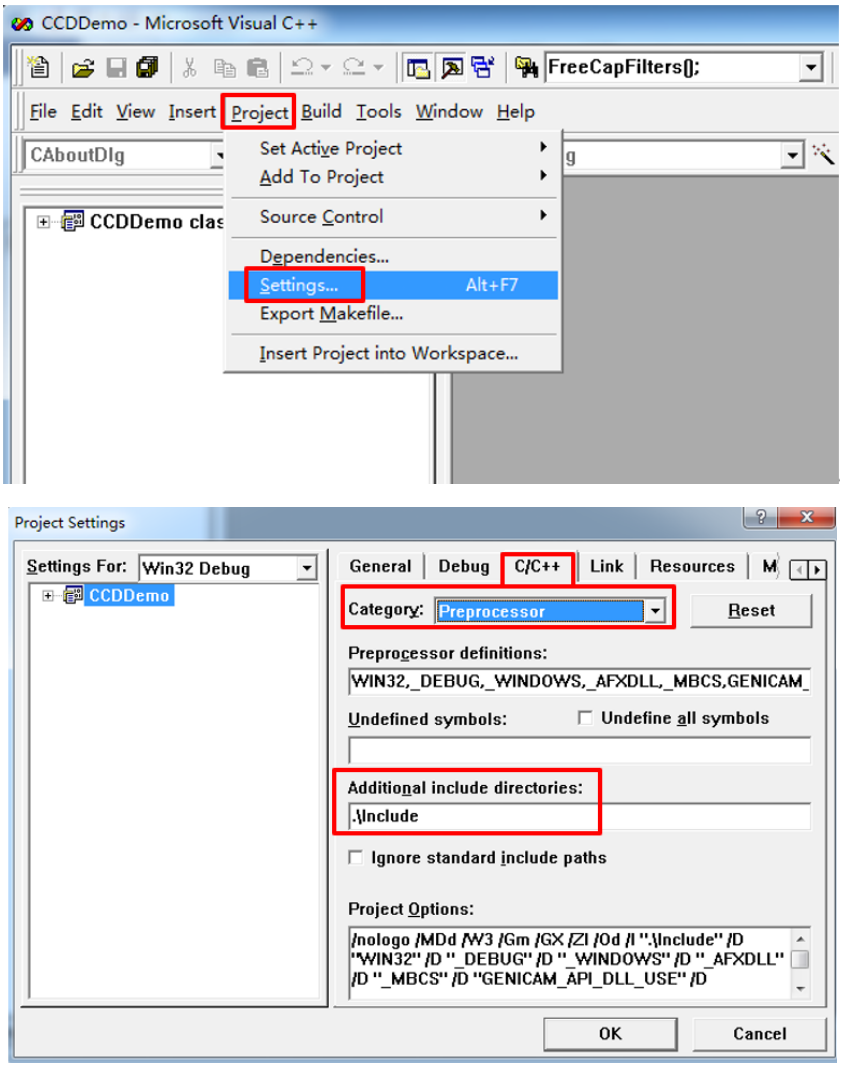
Copy Header File

Figure 1-4 Copy header file



Project Configuration

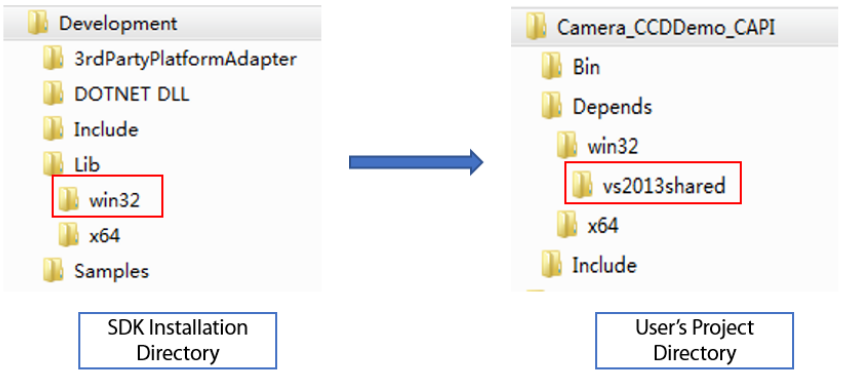
Figure 1-5 Project configuration of header file



1.5.1.2 Configure lib

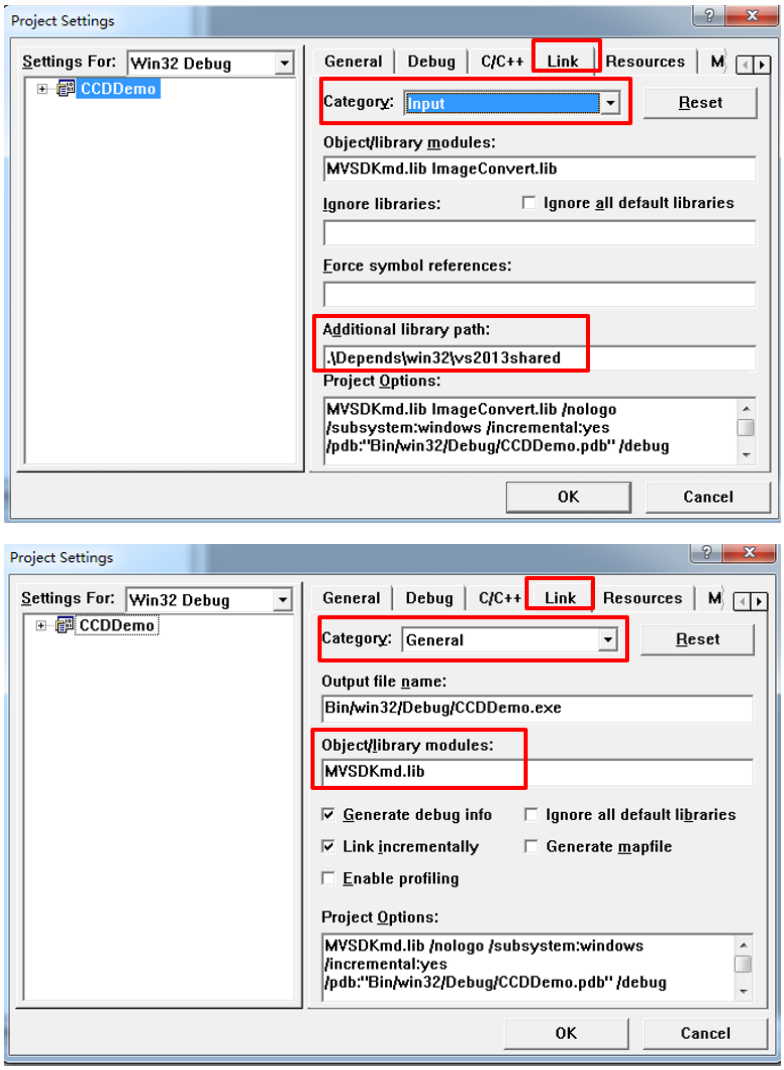
Copy MVSDKmd.lib

Figure 1-6 Copy header file



Project Configuration

Figure 1-7 Project configuration of lib



Runtime Library

No need to copy the .dll library. The corresponding dll library in the path of 【SDK Installation Directory】\Runtime will be used by default.

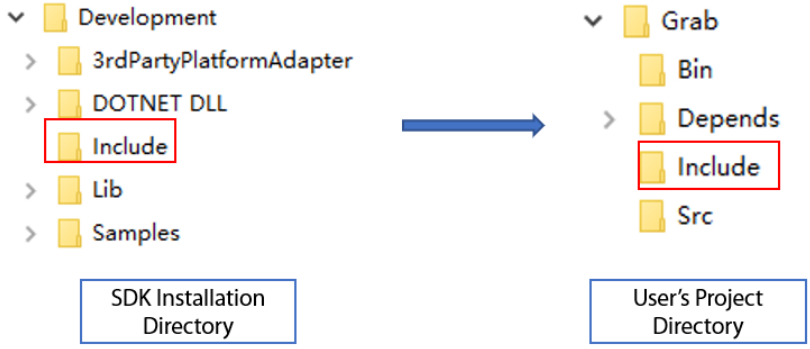
1.5.2 VS2005 ~ VS2013

Take **VS2013 Debug | Win32** as an example to demonstrate the method of project configuration.

1.5.2.1 Configure Header File

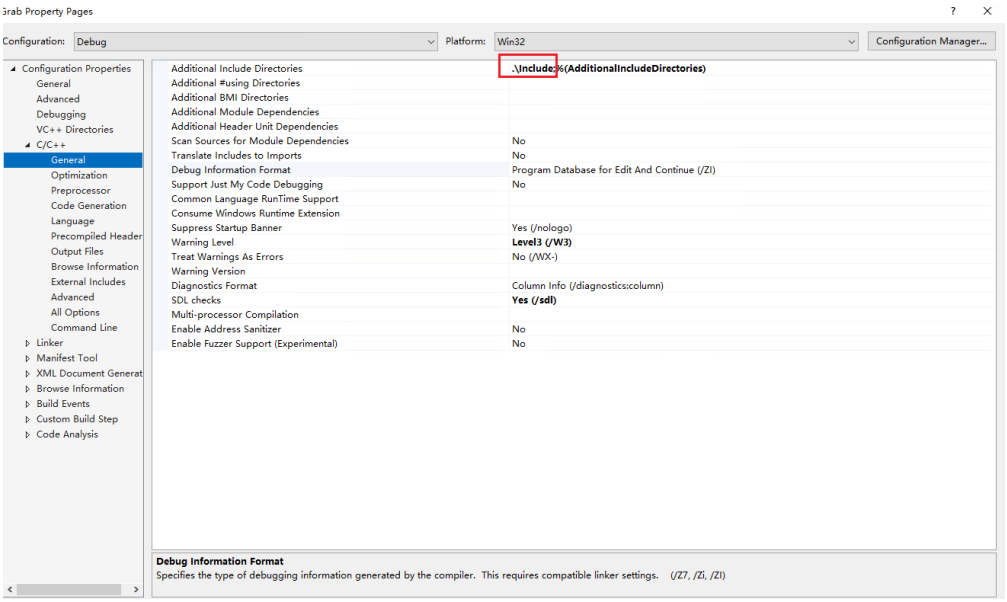
Copy Header File

Figure 1-8 Copy header file



Project Configuration

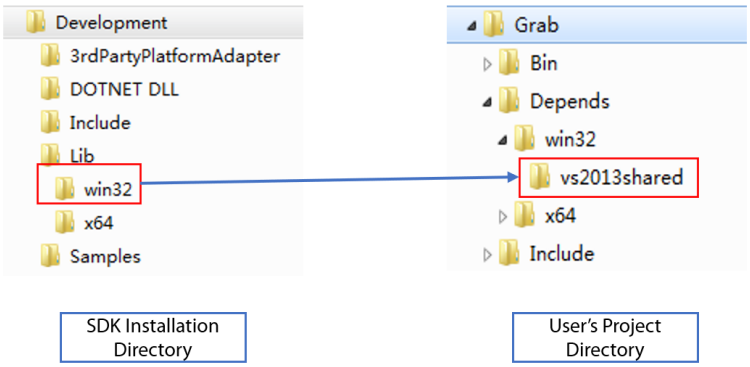
Figure 1-9 Project configuration of header file



1.5.2.2 Configure lib

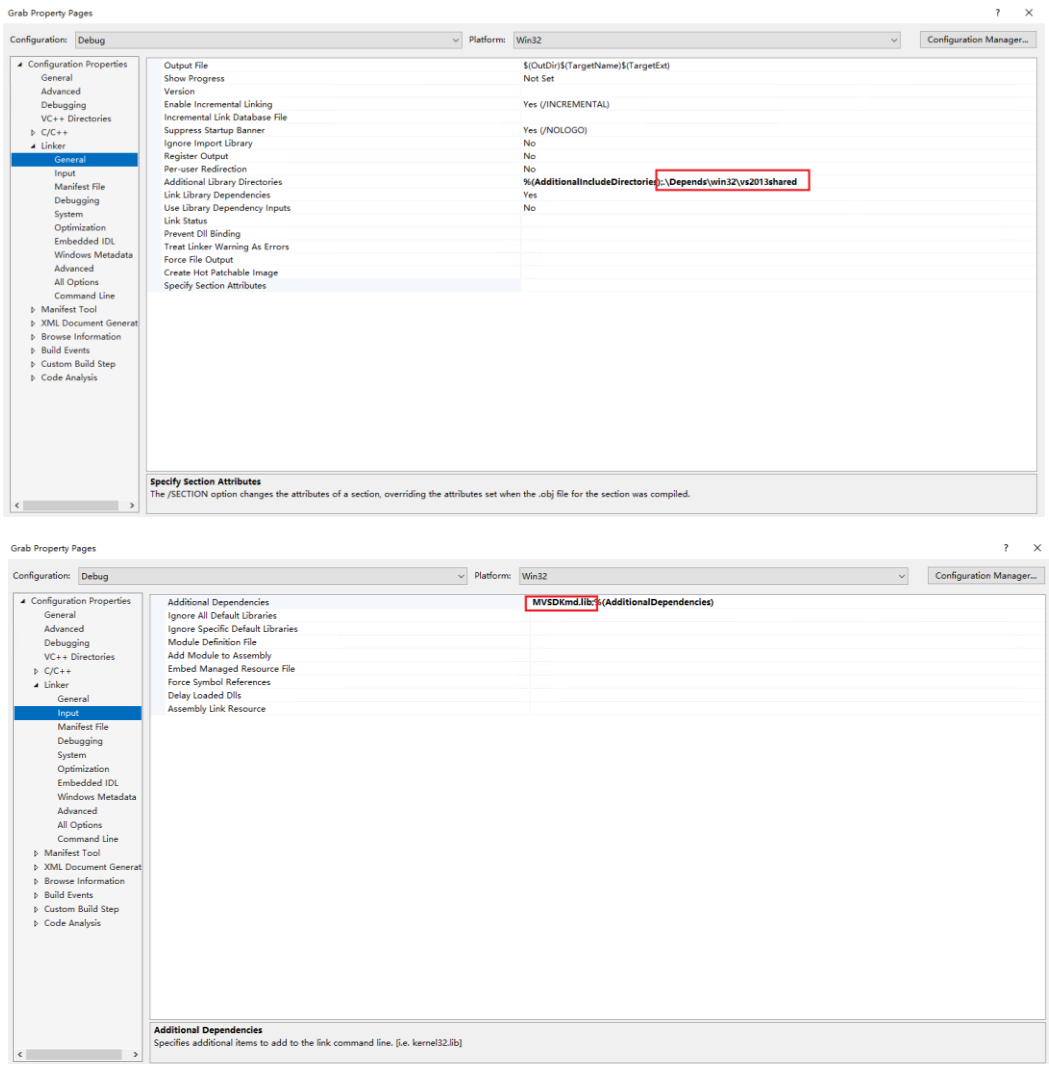
Copy lib

Figure 1-10 Copy lib



Project Configuration

Figure 1-11 Project configuration of lib



Runtime Library

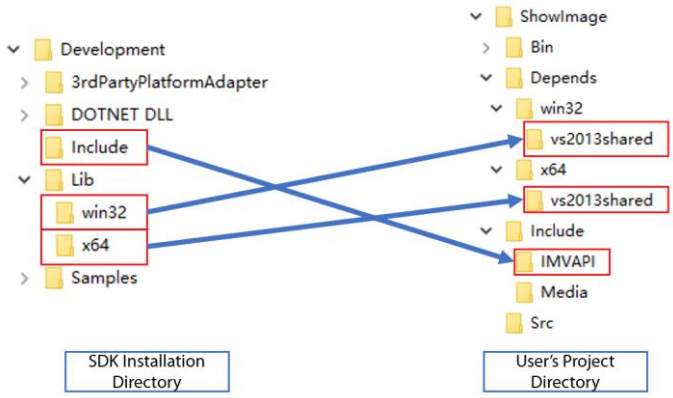
No need to copy the .dll library. The corresponding dll library in the path of 【SDK

Installation Directory】\Runtime will be used by default.

1.5.3 QT (Pro File)

Copy Header File

Figure 1-12 Copy header file



Edit Pro File

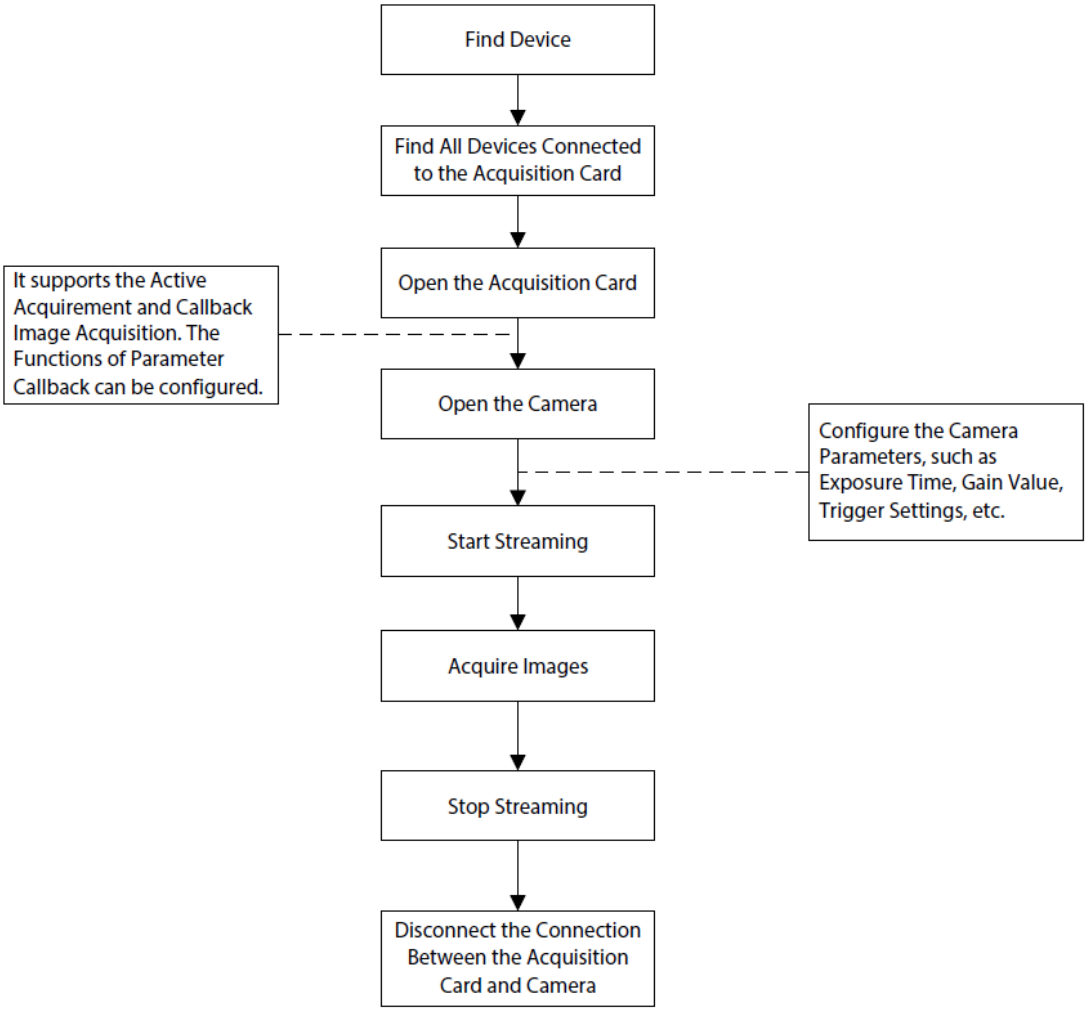
```
Debug {
    contains(QMAKE_COMPILER_DEFINES, _WIN64) {
        LIBS += -L./Depends/x64/vs2013shared                -lMVSDKmd
    }
    else {
        LIBS += -L./Depends/win32/vs2013shared              -lMVSDKmd
    }
}
else {
    contains(QMAKE_COMPILER_DEFINES, _WIN64) {
        LIBS += -L./Depends/x64/vs2013shared                -lMVSDKmd
    }
    else {
        LIBS += -L./Depends/win32/vs2013shared              -lMVSDKmd
      }
}
INCLUDEPATH += ./Include
```

Runtime Library Configuration

Please refer to the runtime library configuration of the 1.4.2 VS2005 ~ 2013.

2 Overall Calling Process of SDK

Figure 2-1 SDK calling process



3 Function Interface Calling of SDK

- The directory of interface header files of SDK:
  - ◇ \Development\Include\IMVFGApi.h
- The directory of the macro definition, enumeration, struct and callback function statement of SDK interface:
  - ◇ \Development\Include\IMVFGDefines.h

Table 3-1 Parameter description

Parameter	Description
IMV_FG_IF_HANDLE	Use the handle of acquisition card.
IMV_FG_DEV_HANDLE	Use the handle of camera.
HANDLE	Use the handle of acquisition card and camera together.

3.1 System Operation Instruction

3.1.1 Obtain Version Information

- Interface: IMV_FG_API const char* IMV_FG_CALL IMV_FG_FG_GetVersion(void);
  - ◇ Description: Get OS version. If succeeded, return version information. If failed, return **NULL**.
  - ◇ Reference Routine: None.

3.1.2 Acquisition Card Enumeration (Find Device)

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_EnumInterface(IN unsigned int nInterfaceType,OUT IMV_FG_INTERFACE_INFO_LIST *pInterfaceList);
  - ◇ Description: Acquisition card enumeration. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-2 Parameter description

Parameter	Description
nInterfaceType [IN]	The interface type to be enumerated can be combined arbitrarily, such as **interfaceTypeGige | interfaceTypeUsb3**.
pInterfaceList [OUT]	Device list.

- When **interfaceType** = **typeGevInterface**, enumerate all online devices connected through the **GigE** interface.
- When **interfaceType** = **typeU3vInterface**, enumerate all online devices connected through the **USB** interface.
- When **interfaceType** = **typeCLInterface**,enumerate the online devices connected through the **CameraLink** interface.
- When **interfaceType** = **typeCXPInterface**, enumerate all online devices connected through the **CXP** interface.
- The **interfaceType** of the IMV_**EnumDevices** support any combination of interface types. For example, if you need to enumerate all online devices connected through the **GigE** port and **USB 3.0** port, the **interfaceType** can be written as the **interfaceType** = **interfaceTypeGige** | **interfaceTypeUsb3**.

◇ Reference Routine: All current routines can be referenced.

3.1.3 Camera Enumeration (Find Device)

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_EnumDevices(IN unsigned int nInterfaceType,OUT IMV_FG_DEVICE_INFO_LIST *pDeviceList);
  - ◇ Description: Enumeration Device. Enumerate the cameras connected to the opened acquisition card.

Table 3-3 Parameter description

Parameter	Description
nInterfaceType [IN]	For the interface type to be enumerated, please refer to the acquisition card interface enumeration.
pDeviceList [OUT]	Device list.

3.2 Acquisition Card Operation

3.2.1 Open and Close Acquisition Card

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_OpenInterface (IN unsigned int nIndex, OUT IMV_FG_IF_HANDLE* handle);
  - ◇ Description: Acquisition card (connect the device according to the handle). If succeeded, return **IMV_FG_OK.** If failed, return error code.

Table 3-4 Parameter description

Parameter	Description
Handle [IN]	Device handle.

◇ Reference Routine: All current routines can be referenced.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_OpenInterfaceEx (IN IMV_FG_ECreateHandleMode mode ,IN void* pIdentifier, OUT IMV_FG_IF_HANDLE* handle);

◇ Description: Acquisition card (connect the device according to the handle). If succeeded, return **IMV_FG_OK.** If failed, return error code.

Table 3-5 Parameter description

Parameter	Description
mode [IN]	Handle creation mode.
pIdentifier [IN]	Open the identifier of the acquisition card.
Handle [OUT]	Device handle.

◇ Reference Routine: **OpenDeviceMethod**

- Interface: IMV_FG_API bool IMV_FG_CALL IMV_FG_IsOpenInterface (IN IMV_FG_IF_HANDLE handle);

◇ Description: Determine whether the acquisition card is opened. If opened, return **True**. If close or offline, return **False**.

Table 3-6 Parameter description

Parameter	Description
Handle [IN]	Device handle.

◇ Reference Routine: All current routines can be referenced.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_CloseInterface (IN IMV_FG_IF_HANDLE handle);

◇ Description: Close Device (Close device according to the handle). If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-7 Parameter description

Parameter	Description
Handle [IN]	Device handle.

◇ Reference Routine: All current routines can be referenced.

3.3 Camera Operation Instruction

3.3.1 Device Open and Close

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_OpenDevice (IN IMV_FG_ECreateHandleMode mode， IN void* pIdentifier， OUT IMV_FG_DEV_HANDLE handle);
  - ◇ Description: Open device. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-8 Parameter description

Parameter	Description
mode [IN]	Handle creation mode.
pIdentifier [IN]	Open the identifier of the camera.
Handle [OUT]	Camera handle.

  - ◇ Reference Routine: All current routines can be referenced.
- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_OpenDeviceEx (IN IMV_FG_ECreateHandleMode mode， IN void* pIdentifier， OUT IMV_FG_DEV_HANDLE handle);
  - ◇ Description: Open device. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-9 Parameter description

Parameter	Description
mode [IN]	Handle creation mode.
pIdentifier [IN]	Open the identifier of the camera.
Handle [OUT]	Camera handle.



It supports the auto-synchronization of camera attributes to the acquisition card function, the attributes include width, height, tap and pixel format.

  - ◇ Reference Routine: All current routines can be referenced.
- Interface: IMV_FG_API bool IMV_FG_CALL IMV_FG_IsDeviceOpen (IN IMV_FG_DEV_HANDLE handle);
  - ◇ Description: Determine whether the device is opened. If opened, return **True**. If close or offline, return **False**.

Table 3-10 Parameter description

Parameter	Description
Handle [IN]	Device handle.

◇ Reference Routine: **ForceIp**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_CloseDevice (IN IMV_FG_DEV_HANDLE handle);

◇ Description: Close Device (Close device according to the handle). If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-11 Parameter description

Parameter	Description
Handle [IN]	Device handle.

◇ Reference Routine: All current routines can be referenced.

3.3.2 Registration Event Callback

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_SubscribeConnectArg(IN IMV_FG_DEV_HANDLE handle, IN IMV_FG_ConnectCallBack proc, IN void* pUser);

◇ Description: Camera connection status event callback registration. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-12 Parameter description

Parameter	Description
Handle [IN]	Camera handle.
proc [IN]	Device connection status event callback function.
pUser [IN]	User-defined data, can be set to **NULL**.

Only one callback function is supported, and after the device is turned off, the registration will be invalid. Re-register the callback function after turning on the device.

◇ Reference Routine: **ResumeConnect**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_SubscribeMsgChannelArg(IN IMV_FG_IF_HANDLE hIFDev, IN IMV_FG_MsgChannelCallBack proc, IN void* pUser);

◇ Description: Notification channel event callback registration. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-13 Parameter description

Parameter	Description
hIFDev [IN]	Device handle.
proc [IN]	Notification channel event callback registration function.
pUser [IN]	User-defined data, can be set to **NULL**.

- The data in the callback function shall be processed promptly, or it will become invalid.
- Only one callback function is supported, and after the device is turned off, the registration will be invalid. Please re-register the callback function after turning on the device.

◇ Reference Routine: **MsgEvents**.

3.4 Streaming Data Operation

3.4.1 Stream Data Configuration

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_SetBufferCount(IN IMV_FG_IF_HANDLE handle, IN unsigned int nSize);
  - ◇ Description: Configure the buffer quantity of frame data. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-14 Parameter description

Parameter	Description
Handle [IN]	Device handle.
nSize [IN]	Buffer quantity.

The configuration cannot be performed during the streaming.

  - ◇ Reference Routine: None.
- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_ClearFrameBuffer(IN IMV_FG_IF_HANDLE handle);
  - ◇ Description: Clear the frame data buffer. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-15 Parameter description

Parameter	Description
Handle [IN]	Device handle.

◇ Reference Routine: **ClearFrameBuffer**.

3.4.2 Image Collection

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_StartGrabbing(IN IMV_FG_IF_HANDLE handle);
  - ◇ Description: Acquiring images. If succeeded, return IMV_FG_OK. If failed, return error code.

Table 3-16 Parameter description

Parameter	Description
Handle [IN]	Device handle.

◇ Reference Routine: **Grab**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_StartGrabbingEx(IN IMV_FG_IF_HANDLE handle, IN uint64_t maxImagesGrabbed, IN IMV_FG_EGrabStrategy strategy);
  - ◇ Description: Acquiring images. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-17 Parameter description

Parameter	Description
Handle [IN]	Device handle.
maxImagesGrabbed [IN]	The max number of frames of acquired images. After the specified number of images is acquired, the acquisition will stop. If the specified number is 0, it means that the acquisition is continuously performed. The parameter of **IMV_FG_StartGrabbing** is 0 by default.
strategy [IN]	Acquisition strategy (**IMV_FG_StartGrabbing** wil use the **grabStrartegySequential** to acquire image).

◇ Reference Routine: **GrabStrategy.**

- Interface: IMV_FG_API bool IMV_FG_CALL IMV_FG_IsGrabbing(IN

IMV_FG_IF_HANDLE handle);

◇ Description: Determine whether the device is streaming. If streaming, return **True**. If not streaming, return **False**.

Table 3-18 Parameter description

Parameter	Description
Handle [IN]	Device handle.

◇ Reference Routine: **MultipleCamera**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_StopGrabbing(IN IMV_FG_IF_HANDLE handle);

◇ Description: Stop streaming. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-19 Parameter description

Parameter	Description
Handle [IN]	Device handle.

◇ Reference Routine: **Grab**.

3.4.3 Acquire Image Data

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_AttachGrabbing(IN IMV_FG_IF_HANDLE handle, IN IMV_FG_FrameCallBack proc, IN void* pUser);

◇ Description: Registered frame data callback function (asynchronous acquisition mechanism for frame data). If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-20 Parameter description

Parameter	Description
Handle [IN]	Device handle.
proc [IN]	Frame data information callback function. Do not handle the time-consuming operations in this function, otherwise, it will affect the real-time performance of subsequent frame data.
pUser [IN]	User-defined data, can be set to **NULL**.



- The asynchronous acquisition mechanism and synchronous acquisition mechanism (**IMV_FG_GetFrame**) are mutually exclusive, therefore, only one of them can be selected in the system for the same device.

- Only one callback function is supported, and after the device is turned off, the registration will be invalid. Please re-register the callback function after turning on the device.

◇ Reference Routine: **FrameClone**, **FrameTriggerCount**, **GrabCallback**, **Grab**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_GetFrame(IN IMV_FG_IF_HANDLE handle, OUT IMV_FG_Frame* pFrame, IN unsigned int timeoutMS);

◇ Description: Receive a frame of image (Synchronous acquisition mechanism for frame data). If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-21 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFrame [OUT]	The frame data information.
timeoutMS[IN]	The timeout value for obtaining a frame of image. The INFINITE mode means that it will continuously wait until a frame of image is received, otherwise, it will stop acquiring (unit: ms).



- This interface does not support the multi-thread callings.
- The synchronous and asynchronous acquisition mechanism (**IMV_FG_AttachGrabbing**) are mutually exclusive, therefore, only one of them can be selected in the system for the same device.
- Acquiring images through releasing the internal cache.
- Use **IMV_FG_ReleaseFrame** to release the image buffer space.

◇ Reference Routine: **Grab**

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_ReleaseFrame(IN IMV_FG_IF_HANDLE handle, IN IMV_FG_Frame* pFrame);

◇ Description: Release the image buffer space. If succeeded, return **IMV_FG_OK.** If failed, return error code.

Table 3-22 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFrame [IN]	The frame data information.

◇ Reference Routine: **Grab**.

3.4.4 Operations on Streaming and Image Data

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_CloneFrame(IN IMV_FG_IF_HANDLE handle, IN IMV_FG_Frame* pFrame, OUT IMV_FG_Frame* pCloneFrame);
  - ◇ Description: Frame data deep copy and clone. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-23 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFrame [IN]	Clone the source frame data information.
pCloneFrame [OUT]	New frame data information.



Use **IMV_FG_ReleaseFrame** to release the image buffer space.

  - ◇ Reference Routine: **FrameClone**
- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_GetStatisticsInfo(IN IMV_FG_IF_HANDLE handle, OUT IMV_FG_StreamStatisticsInfo* pStreamStatsInfo);
  - ◇ Description: Obtain statistics information of streaming (after performing **IMV_FG_StartGrabbing** / **IMV_FG_StartGrabbing** to call the obtaining command). If succeeded, return **IMV_FG_OK.** If failed, return error code.

Table 3-24 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pStreamStatsInfo [OUT]	Stream statistics information.

  - ◇ Reference Routine: **StreamStatistics**.
- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_ResetStatisticsInfo(IN IMV_FG_IF_HANDLE handle);
  - ◇ Description: Reset statistics information of streaming (after performing **IMV_FG_StartGrabbing** / **IMV_FG_StartGrabbing** to call the resetting command). If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-25 Parameter description

Parameter	Description
Handle [IN]	Device handle.

  - ◇ Reference Routine: **StreamStatistics**.

3.5 Attribute Operation Instruction

3.5.1 Configuration Download, Storage and Loading

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_DownLoadGenICamXML(IN HANDLE handle, IN const char* pFullFileName);
  - ◇ Description: Download the device description XML file, and save it to the specified path, such as **D:\\xml.zip**. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-26 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFullFileName[IN]	File saving path.

  - ◇ Reference Routine: None.
- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_SaveDeviceCfg(IN HANDLE handle, IN const char* pFullFileName);
  - ◇ Description: Saving the device configuration file to the specified folder. The file with the same name that already exists will be overwritten. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-27 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFullFileName [IN]	Full name (including saving path) of the exported device configuration file, such as **D:\\config.xml** or **D:\\config.mvcfg**.

  - ◇ Reference Routine: **LoadAndSaveDeviceCfg.**
- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_LoadDeviceCfg(IN HANDLE handle, IN const char* pFullFileName, OUT IMV_FG_ErrorList* pErrorList);
  - ◇ Description: Load the xml configuration of device from files. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-28 Parameter description

Parameter	Description
Handle [IN]	Device handle.

pFullFileName [IN]	Full name (including saving path) of the device configuration file (XML), such as **D:\\config.xml** or **D:\\config.mvcfg**.
pErrorList [OUT]	The list of attribute name that failed to load. The upper limit of attributes that fails to save and load is **IMV_FG_MAX_ERROR_LIST_NUM**.

◇ Reference Routine: **LoadAndSaveDeviceCfg**.

3.5.2 Attribute Determination, Acquisition and Setting

3.5.2.1 Attribute Determination

- Interface: IMV_FG_API bool IMV_FG_CALL IMV_FG_FeatureIsAvailable(IN HANDLE handle, IN const char* pFeatureName);
  - ◇ Description: Determine whether the attribute is available. If available, return **True**. If unavailable, return **False**.

Table 3-29 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.

  - ◇ Reference Routine: **CommPropAccess**.
- Interface: IMV_FG_API bool IMV_FG_CALL IMV_FG_FeatureIsReadable (IN HANDLE handle, IN const char* pFeatureName);
  - ◇ Description: Determine whether the attribute is readable. If readable, return **True**. If unreadable, return **False**.

Table 3-30 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.

  - ◇ Reference Routine: **CommPropAccess**.
- Interface: IMV_FG_API bool IMV_FG_CALL IMV_FG_FeatureIsWriteable (IN HANDLE handle, IN const char* pFeatureName);
  - ◇ Description: Determine whether the attribute is writable. If writable, return **True**. If unwritable, return **False**.

Table 3-31 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.

◇ Reference Routine: **CommPropAccess**

- Interface: IMV_FG_API bool IMV_FG_CALL IMV_FG_FeatureIsStreamable (IN HANDLE handle, IN const char* pFeatureName);
  - ◇ Description: Determine whether the attribute is streamable. If streamable, return **True**. If unstreamable, return **False**.

Table 3-32 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.

◇ Reference Routine: **CommPropAccess.**

- Interface: IMV_FG_API bool IMV_FG_CALL IMV_FG_FeatureIsValid (IN HANDLE handle, IN const char* pFeatureName);
  - ◇ Description: Determine whether the attribute is valid. If valid, return **True**. If invalid, return **False**.

Table 3-33 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.

◇ Reference Routine: **CommPropAccess**

3.5.2.2 Attribute Acquisition

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_GetIntFeatureValue(IN HANDLE handle, IN const char* pFeatureName, OUT int64_t* pIntValue);
  - ◇ Description: Obtain the integer attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-34 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.

pIntValue [OUT]	The attribute value of Integer.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_GetIntFeatureMin (IN HANDLE handle, IN const char* pFeatureName, OUT int64_t* pIntValue);

  ◇ Description: Obtain the configurable min value of the integer attribute. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-35 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The min value that the integer can be configured.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_GetIntFeatureMax (IN HANDLE handle, IN const char* pFeatureName, OUT int64_t* pIntValue);

  ◇ Description: Obtain the configurable max value of integer attribute. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-36 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The configurable max value of the integer attribute.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_GetIntFeatureInc (IN HANDLE handle, IN const char* pFeatureName, OUT int64_t* pIntValue);

  ◇ Description: Obtain the step size of the integer attribute. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-37 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	Integer attribute step size.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_GetDoubleFeatureValue (IN HANDLE handle, IN const char* pFeatureName, OUT double* pDoubleValue);

  ◇ Description: Obtain the floating-point attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-38 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The attribute value of floating point.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_GetDoubleFeatureMin (IN HANDLE handle, IN const char* pFeatureName, OUT double* pDoubleValue);

  ◇ Description: Obtain the configurable min value of the floating-point attribute. If succeeded, return **IMV_FG_OK.** If failed, return error code.

Table 3-39 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The configurable min value of the floating-point attribute.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_GetDoubleFeatureMax (IN HANDLE handle, IN const char* pFeatureName, OUT double* pDoubleValue);

  ◇ Description: Obtain the configurable max value of floating-point attribute. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-40 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The configurable max value of the floating-point attribute.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_GetBoolFeatureValue (IN HANDLE handle, IN const char* pFeatureName, OUT bool* pBoolValue);

  ◇ Description: Obtain the Boolean attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-41 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.

pIntValue [OUT]	The attribute value of Boolean.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_GetStringFeatureValue (IN HANDLE handle, IN const char* pFeatureName, OUT IMV_FG_String* pStringValue);

◇ Description: Obtain the string attribute value. If succeeded, return IMV_FG_OK. If failed, return error code.

Table 3-42 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pIntValue [OUT]	The attribute value of String.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_GetEnumFeatureValue (IN HANDLE handle, IN const char* pFeatureName, OUT uint64_t* pEnumValue);

◇ Description: Obtain the enumeration attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-43 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pEnumValue [OUT]	The attribute value of enumeration.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_GetEnumFeatureSymbol (IN HANDLE handle, IN const char* pFeatureName, OUT IMV_FG_String* pEnumValueSymbol);

◇ Description: Obtain the enumeration symbol value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-44 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pEnumValue [OUT]	The symbol value of enumeration.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_GetEnumFeatureEntryNum (IN HANDLE handle, IN const char* pFeatureName, OUT unsigned int* pEntryNum);

◇ Description: Obtain the configurable number of the enumeration attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-45 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pEntryNum [OUT]	The configurable quantity of the enumeration attribute value.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_GetEnumFeatureEntrys (IN HANDLE handle, IN const char* pFeatureName, OUT IMV_FG_EnumEntryList* pEnumEntryList);

◇ Description: Obtain the enumeration value list of the enumeration attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-46 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pEnumEntryList [OUT]	The list of configurable enumeration value of enumeration attribute.

◇ Reference Routine: **CommPropAccess**.

3.5.2.3 Attribute Setting

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_SetIntFeatureValue(IN HANDLE handle, IN const char* pFeatureName, IN int64_t intValue);

◇ Description: Obtain the integer attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-47 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
intValue [IN]	The integer attribute value to be configured.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_SetDoubleFeatureValue (IN HANDLE handle, IN const char* pFeatureName, IN double doubleValue);

◇ Description: Configure the floating-point attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-48 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
doubleValue [IN]	Floating point attribute value which is to be configured.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_SetBoolFeatureValue (IN HANDLE handle, IN const char* pFeatureName, IN bool boolValue);

◇ Description: Configure the Boolean attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-49 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
boolValue [IN]	Boolean attribute value which is to be configured.

◇ Reference Routine: **ChunkData**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_SetEnumFeatureValue (IN HANDLE handle, IN const char* pFeatureName, IN uint64_t enumValue);

◇ Description: Configure the enumeration attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-50 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
enumValue [IN]	Enumeration attribute value which is to be configured.

◇ Reference Routine: **CommPropAccess**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_SetEnumFeatureSymbol (IN HANDLE handle, IN const char* pFeatureName, IN const char* pEnumSymbol);

◇ Description: Configure the enumeration symbol value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-51 Parameter description

Parameter	Description

Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
boolValue [IN]	The symbol value of enumeration which is to be configured.

◇ Reference Routine: **CommPropAccess**, **SoftTrigger**, **UserSetControl**

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_SetStringFeatureValue (IN HANDLE handle, IN const char* pFeatureName, IN const char* pStringValue);

◇ Description: Configure the string attribute value. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-52 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.
pStringValue [IN]	String attribute value which is to be configured.

◇ Reference Routine: **CommPropAccess**.

3.5.3 Execution of Command Attribute

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_ExecuteCommandFeature(IN HANDLE handle, IN const char* pFeatureName);

◇ Description: Execute the command attribute. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-53 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pFeatureName [IN]	Attribute name.

◇ Reference Routine: **ClearFrameBuffer**, **SoftTrigger**, **UserSetControl**.

3.5.4 Attribute Search and Usage

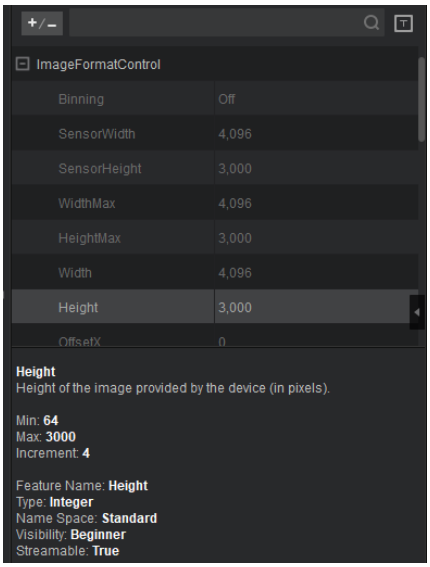
User can search the attribute on the client, and learn the detailed information of the attribute, such as data type, value range, etc.

3.5.4.1 Integer Attribute

If user needs to obtain the pixel height information of the camera, find and click the Height

Attribute. The attribute information of it will be displayed in the below. The data type of the Height Attribute is integer as shown in the figure.

Figure 3-1 Attributes of height



User can obtain and modify the found attributes.

```
    int ret = IMV_FG_OK;
  int64_t heightValue = 0;
  int64_t heightMinValue = 0;
  int64_t heightMaxValue = 0;
  int64_t incrementValue = 0;

  // Obtain attribute value.
  ret = IMV_FG_GetIntFeatureValue(devHandle, "Height", &heightValue);
  if (IMV_FG_OK != ret)
  {
      printf("Get feature value failed! ErrorCode[%d]\n", ret);
      return ret;
  }

  // Obtain the configurable min value of attribute. If there is no min value, return the error code.
  ret = IMV_FG_GetIntFeatureMin(devHandle, "Height", &heightMinValue);
  if (IMV_FG_OK != ret)
  {
      printf("Get feature minimum value failed! ErrorCode[%d]\n", ret);
```

```
        return ret;
    }

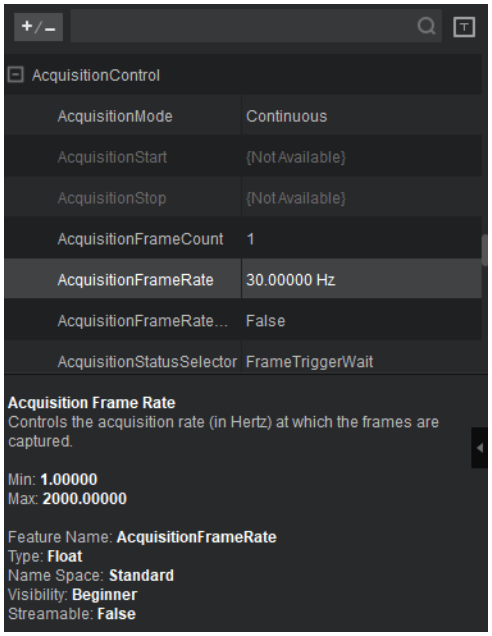
    // Obtain the configurable max value of attribute.If there is no max value, return the
error code.
    ret = IMV_FG_GetIntFeatureMax(devHandle, "Height", &heightMaxValue);
    if (IMV_FG_OK != ret)
    {
        printf("Get feature maximum value failed! ErrorCode[%d]\n", ret);
        return ret;
    }

    // Obtain the attribute step size. If there is no step-size value, return the error code.
    ret = IMV_FG_GetIntFeatureInc(devHandle, "Height", &incrementValue);
    if (IMV_FG_OK != ret)
    {
        printf("Get feature increment value failed! ErrorCode[%d]\n", ret);
        return ret;
    }

    // Configure attribute value.
    ret = IMV_FG_SetIntFeatureValue(devHandle, "Height", heightValue);
    if (IMV_FG_OK != ret)
    {
        printf("Set feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }
```

3.5.4.2 Floating Point Attribute

Figure 3-2 Attributes of floating point




```
    int ret = IMV_FG_OK;
    float framerate = 0;
    float frameratemin = 0;
    float frameratemax = 0;

    // Obtain attribute value.
    ret = IMV_FG_GetDoubleFeatureValue (devHandle, "AcquisitionFrameRate",
&framerate);
    if (IMV_FG_OK != ret)
    {
        printf("Get feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }
    // Obtain the configurable min value of attribute. If there is no min value, return the
error code.
    ret = IMV_FG_GetDoubleFeatureMin(devHandle, "AcquisitionFrameRate",
&frameratemin);
    if (IMV_FG_OK != ret)
    {
        printf("Get feature minimum value failed! ErrorCode[%d]\n", ret);
```

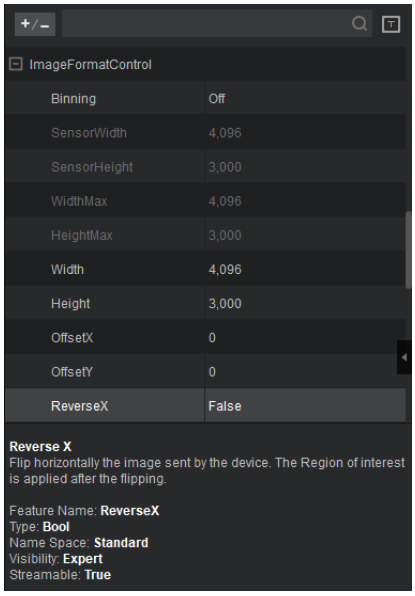
```
        return ret;
    }

    // Obtain the configurable max value of attribute.If there is no max value, return the
error code.
    ret = IMV_FG_GetDoubleFeatureMax(devHandle, "AcquisitionFrameRate",
&frameratemax);
    if (IMV_FG_OK != ret)
    {
        printf("Get feature maximum value failed! ErrorCode[%d]\n", ret);
        return ret;
    }

    // Configure attribute value.
    ret = IMV_FG_SetDoubleFeatureValue(devHandle, "AcquisitionFrameRate",
framerate);
    if (IMV_FG_OK != ret)
    {
        printf("Set feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }
```

3.5.4.3 Boolean Attribute

Figure 3-3 Attributes of Boolean



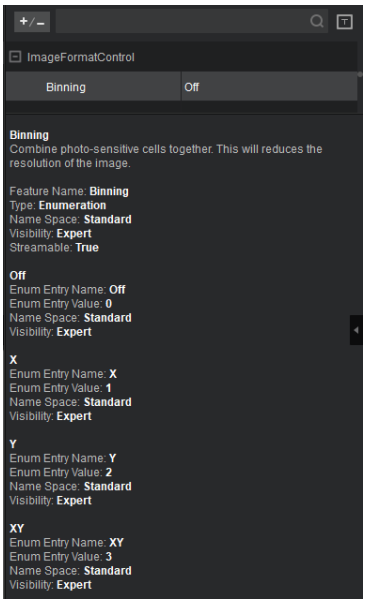

```
  int ret = IMV_FG_OK;
bool reversevalue = 0;

// Obtain attribute value.
ret = IMV_FG_GetBoolFeatureValue (devHandle, "ReverseX", &reversevalue);
if (IMV_FG_OK != ret)
{
        printf("Get feature value failed! ErrorCode[%d]\n", ret);
        return ret;
}

// Configure attribute value.
ret = IMV_FG_SetBoolFeatureValue(devHandle, "ReverseX", reversevalue);
if (IMV_FG_OK != ret)
{
        printf("Set feature value failed! ErrorCode[%d]\n", ret);
        return ret;
}
```

3.5.4.4 Enumeration Attribute

Figure 3-4 Attributes of enumeration




```
  int ret = IMV_FG_OK;
uint64_t enumValue;
  IMV_FG_String pEnumSymbol;
  unsigned int pEntryNum;
  IMV_FG_EnumEntryList pEnumEntryList;

// Obtain the configurable enumeration value list of attribute.
ret = IMV_FG_GetEnumFeatureValue(devHandle, "Binning", &pEnumEntryList);
if (IMV_FG_OK != ret)
{
      printf("Get feature value failed! ErrorCode[%d]\n", ret);
      return ret;
}

//Obtain the number of configurable enumeration values of attribute
ret = IMV_FG_GetEnumFeatureEntryNum(devHandle, "Binning", &pEntryNum);
if (IMV_FG_OK != ret)
{
      printf("Get feature value failed! ErrorCode[%d]\n", ret);
      return ret;
```

```
    }

    // Obtain the attribute symbol value.
    ret = IMV_FG_GetEnumFeatureSymbol(devHandle, "Binning", &pEnumSymbol);
    if (IMV_FG_OK != ret)
    {
        printf("Get feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }

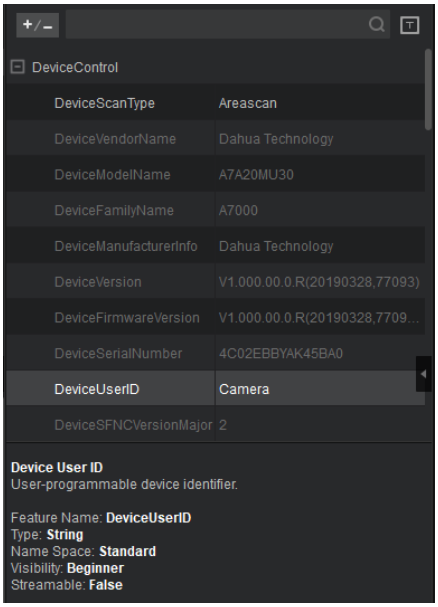
    // Obtain attribute value.
    ret = IMV_FG_GetIntFeatureInc(devHandle, "Binning", &enumValue);
    if (IMV_FG_OK != ret)
    {
        printf("Get feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }

    // Configure attribute symbol value.
    ret = IMV_FG_SetEnumFeatureValue(devHandle, "Binning", pEnumSymbol);
    if (IMV_FG_OK != ret)
    {
        printf("Set feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }

// Configure attribute value.
    ret = IMV_FG_SetEnumFeatureValue(devHandle, "Binning", enumValue);
    if (IMV_FG_OK != ret)
    {
        printf("Set feature value failed! ErrorCode[%d]\n", ret);
        return ret;
    }
```

3.5.4.5 String Attribute

Figure 3-5 Attributes of String



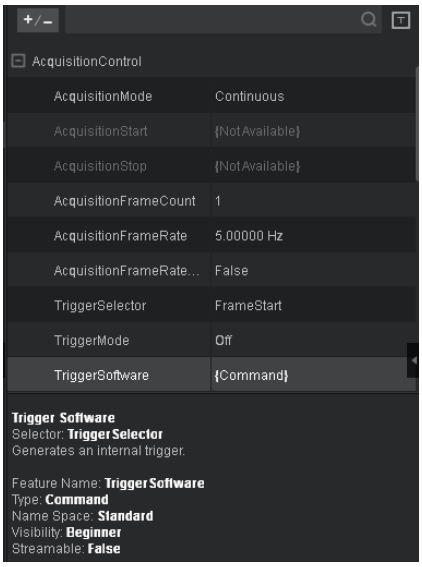

```
  int ret = IMV_FG_OK;
string id = "";

// Obtain attribute value.
ret = IMV_FG_GetStringFeatureValue (devHandle, "DeviceUserID", &id);
if (IMV_FG_OK != ret)
{
    printf("Get feature value failed! ErrorCode[%d]\n", ret);
    return ret;
}

// Configure attribute value.
ret = IMV_FG_SetStringFeatureValue(devHandle, "DeviceUserID", id);
if (IMV_FG_OK != ret)
{
    printf("Set feature value failed! ErrorCode[%d]\n", ret);
    return ret;
}
```

3.5.4.6 Command Attribute

Figure 3-6 Attributes of command




```
int ret = IMV_FG_OK;
// Execute the attribute value of command.
ret = IMV_FG_ExecuteCommandFeature (devHandle, "TriggerSoftware");
if (IMV_FG_OK != ret)
{
    printf("Execute Command feature failed! ErrorCode[%d]\n", ret);
    return ret;
}
```

3.6 Image Operation

3.6.1 Video Recording

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_OpenRecord(IN IMV_FG_IF_HANDLE handle, IN IMV_FG_RecordParam *pstRecordParam);
  - ◇ Description: Start recording. If succeeded, return **IMV_FG_OK.** If failed, return error code.

Table 3-54 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pstRecordParam[IN]	Recording parameter struct.

◇ Reference Routine: **Record**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_InputOneFrame(IN IMV_FG_IF_HANDLE handle, IN IMV_FG_RecordFrameInfoParam *pstRecordFrameInfoParam);

  ◇ Description: Record a frame of image. If succeeded, return **IMV_FG_OK**. If failed, return error code.

  Table 3-55 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pstRecordFrameInfoParam [IN]	Frame information structure for recording.

  ◇ Reference Routine: **Record**.

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_CloseRecord(IN IMV_FG_IF_HANDLE handle);

  ◇ Description: Stop recording. If succeeded, return **IMV_FG_OK**. If failed, return error code.

  Table 3-56 Parameter description

Parameter	Description
Handle [IN]	Device handle.

  ◇ Reference Routine: **Record**.

3.6.2 Image Operation

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_PixelConvert(IN IMV_FG_IF_HANDLE handle, IN_OUT IMV_FG_PixelConvertParam* pstPixelConvertParam);

  ◇ Description: Pixel format conversion. If succeeded, return **IMV_FG_OK**. If failed, return error code.

  Table 3-57 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pstPixelConvertParam [IN][OUT]	Pixel format conversion parameter structure.

  □ The supported target pixel format: **gvspPixelRGB8 /gvspPixelBGR8 /gvspPixelMono8 /gvspPixelBGRA8.**

- Convert the raw image data into the pixel format required by the user and stored in the specified memory by the caller.
- When pixel format is **YUV411Packed**, the image width must be divisible by 4.
- When pixel format is **YUV422Packed**, the image width must be divisible by 2.
- When pixel format is **YUYVPacked**, the image width must be divisible by 2.
- Converted image: Data storage starts from the first row at the top, which is the default storage direction of camera data.

◇ Reference Routine: **ImageConvert**, **SaveImageToBmp.**

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_FlipImage(IN IMV_FG_IF_HANDLE handle, IN_OUT IMV_FG_FlipImageParam* pstFlipImageParam);

◇ Description: Image flip. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-58 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pstFlipImageParam [IN][OUT]	Image flip parameter structure.

- The supported pixel format of the vertical and horizontal image flip: **gvspPixelRGB8** /**gvspPixelBGR8** /**gvspPixelMono8**.
- Convert the raw image data and store them in the specified memory by the caller.

◇ Reference Routine: **ImageFlip**

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_RotateImage(IN IMV_FG_IF_HANDLE handle, IN_OUT IMV_FG_RotateImageParam* pstRotateImageParam);

◇ Description: Clockwise rotation of the image. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-59 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pstRotateImageParam[IN][OUT]	Image rotation parameter structure.

- The supported image format of the 90/180/270 degrees clockwise rotation: **gvspPixelRGB8** / **gvspPixelBGR8** / **gvspPixelMono8**.

- Rotate the raw image data and store them in the specified memory by the caller.

◇ Reference Routine: **ImageRotate.**

- Interface: IMV_FG_API int IMV_FG_CALL IMV_FG_SaveImageToFile (IN IMV_FG_IF_HANDLE handle, IN IMV_FG_SaveImageToFileParam* pstSaveFileParam);

◇ Description: Save the image into the file. If succeeded, return **IMV_FG_OK**. If failed, return error code.

Table 3-60 Parameter description

Parameter	Description
Handle [IN]	Device handle.
pstSaveFileParam [IN]	Save-image-to-file parameter structure.

- The image format supported by the interface includes BMP/JPEG/PNG/TIFF.
- The supported max height-width size of JPEG format is 65,500.

◇ Reference Routine: **SaveImageToFile.**

4 Image Format of 3rd-party Platform Conversion

4.1 Halcon

4.1.1 Mono8 (Monochrome Image)

When image format is Mono8, it can be directly converted to Halcon image object.

It is assumed here that the frame is the frame data struct of **IMV_FG_Frame** acquired by SDK.

```
Hobject grabImg; //Halcon image object
gen_image1_extern(&(grabImg),
                  "byte",
                  (Hlong) frame.frameInfo.width,
                  (Hlong) frame.frameInfo.height,
                  (Hlong)( frame.pData ),
                  0);
```

4.1.2 Non-Mono8 (Color Image)

Non-Mono8 format image shall be converted into BGR.

Please refer to the **Chapter 3.6.2 Image Operation**.

It is assumed here that the frame is the frame data struct of **IMV_FG_Frame** acquired by SDK and the **pBGRbuffer** is the BGR data after conversion.

```
Hobject grabImg; //Halcon image object
gen_image_interleaved(&grabImg,
                      (Hlong) pBGRbuffer,
                      "bgr", frame.frameInfo.width,
                      frame.frameInfo.height, 0,
                      "byte", frame.frameInfo.width,
                      frame.frameInfo.height,0,0,8,0);
```

4.2 OpenCV Object

4.2.1 Mono8 (Monochrome Image)

When image format is Mono8, it can be directly converted to Halcon image object.

It is assumed here that the frame is the frame data struct of **IMV_FG_Frame** acquired by SDK.

```
//Convert to the object of cv::Mat
cv::Mat image = cv::Mat(frame.frameInfo.height,
                        frame.frameInfo.width,
                        CV_8U,
                            (uint8_t*)(( frame.pData));
```

4.2.2 Non-Mono8 (Color Image)

Non-Mono8 format image shall be converted into BGR.

Please refer to the **Chapter 3.6.2 Image Operation**.

It is assumed here that the frame is the frame data struct of **IMV_FG_Frame** acquired by SDK and the **pBGRbuffer** is the BGR data after conversion.

```
//Convert to the object of cv::Mat
Mat image = cv::Mat(frame.frameInfo.height,
                    frame.frameInfo.width,
                    CV_8UC3,
                    (uint8_t*)pBGRbuffer);
```

4.3 QT Object

4.3.1 Mono8 (Monochrome Image)

When image format is Mono8, it can be directly converted to Halcon image object.

It is assumed here that the frame is the frame data struct of **IMV_FG_Frame** acquired by SDK.

```
QImage image; //Qimage object of Qt
image = QImage(frame.pData,
                frame.frameInfo.width,
                frame.frameInfo.height,
                QImage::Format_Grayscale8);
```

4.3.2 Non-Mono8 (Color Image)

Non-Mono8 format image shall be converted into BGR.

Please refer to the **Chapter 3.6.2 Image Operation**.

It is assumed here that the frame is the frame data struct of **IMV_FG_Frame** acquired by SDK and the **pBGRbuffer** is the BGR data after conversion.

```
QImage image; //Qimage object of Qt
image = QImage((uint8_t*) pBGRbuffer,
                frame.frameInfo.width,
                frame.frameInfo.height,
                QImage::Format_RGB888);
```

4.4 MvpImage Object

4.4.1 Mono8 (Monochrome Image)

For formats such as Mono10 and Mono12, refer to section 3.6.2 for converting the image format to Mono8. When the camera's image format is Mono8, the captured image data can be directly used to construct the object.

It is assumed here that the **frame** is the frame data struct of **IMV_FG_Frame** acquired by SDK.

```
MvpImage mvpimage;
mvpCreateEmptyImage(&mvpimage);//init
mvpGenImage1(&mvpimage,
                MVP_IMGDTP_U8,
                frame.frameInfo.height,
                frame.frameInfo.width,
                frame.pData);
```

```
mvpReleaseImage(&mvpimage);          //need release
```

4.4.2 Non-Mono8 (Color Image)

Non-Mono8 format image shall be converted into **BGR** firstly.

It is assumed here that the frame is the frame data struct of **IMV_FG_Frame** acquired by SDK and the **pBGRbuffer** is the BGR data after conversion.

Please refer to the **Chapter 3.6.2 Image Operation** in details.

```
MvpImage mvpimage;
mvpCreateEmptyImage(&mvpimage);//init
mvpGenImageInterleaved（&mvpimage，
                        pBGRbuffer，
                        MVP_COLOR_SPACE_BGR，
                        frame.frameInfo.width,
                        frame.frameInfo.height,
                        0
                        MVP_IMGDTP_U8，
                        frame.frameInfo.width,
                        frame.frameInfo.height,
                        0，
                        0，
                        -1，
                        ,0）;
mvpReleaseImage(&mvpimage);          //need release
```

5 Matched Routine Description

C Routine Path:【SDK Installation Directory】\Development\Samples\VC\IMVFG.

Routine Name	Routine Description
CommPropAccess	Common attribute routine.
FrameClone	Clone frame.
FrameTriggerCount	Frame trigger statistic.
Grab	Image capturing in free-run mode (active streaming).
GrabAsynchronous	Asynchronous frame acquisition.
GrabCallback	Streaming callback.
GrabStrategy	Image acquisition strategy.
ImageConvert	Image format conversion.
ImageFlip	Image flipping.
ImageRotate	Image rotation.
LineTrigger	Image acquisition in external trigger mode (callback to acquire).
LineTriggerEncoder	Image acquisition in encoder method of the external trigger mode.
LoadAndSaveDeviceCfg	Save and load the attribute configuration file of device.
MsgEvents	Registration message event.
MultiCameraGrab	Multi-camera calling routine using C interface in a class-encapsulation.
OpenDeviceMethod	Multi-method of handle creation to open the device routine.
SaveImageToBmp	Save the BMP image.
SaveImageToFile	Save image to the PC.
SoftTrigger	Acquisition in software trigger mode (callback to acquire image).
StreamStatistics	Image acquisition statistics.

6 FAQ (Frequently Asked Question)

6.1 Only 8 Images are Obtained by Using the C Interface

Reason

After acquiring the images using C interface, the images are not released resulting in the buffer memory being filled and preventing the acquisition of new images.

Solution

Every time the frame is acquired, the buffer memory shall be release by calling the interface of **IMV_FG_ReleaseFrame** as shown in the figure below.

Figure 6-1 IMV_ReleaseFrame calling

```
// Get a frame image
ret = IMV_FG_GetFrame(camHandle, &frame, 500);
if (IMV_FG_OK != ret)
{
    printf("Get frame failed! ErrorCode[%d]\n", ret);
    continue;
}
```

```
// Free image buffer
ret = IMV_FG_ReleaseFrame(camHandle, &frame);
if (IMV_FG_OK != ret)
{
    printf("Release frame failed! ErrorCode[%d]\n", ret);
}
```